



TRABAJO INTEGRADOR FINAL

Objetivo

El propósito de este trabajo es crear de forma grupal un juego programado en C que ponga en práctica los temas vistos durante la cursada. Más allá de lograr una aplicación funcional, es fundamental que cada miembro del equipo sea capaz de argumentar las decisiones de diseño adoptadas, rastrear la ejecución de una jugada a lo largo del código y fundamentar la estructura general del proyecto.

1. Integración y organización de los grupos

Conformación de equipos (de 2 a 4 alumnos): Los grupos deberán reportar los nombres de sus miembros junto con la propuesta de juego seleccionada, aguardando la validación formal de la cátedra (la cual se reserva el derecho de solicitar o disponer una reasignación en caso de ser requerido).

Lineamientos para la dinámica de trabajo:

- Repartir las responsabilidades de manera equitativa entre todos los participantes. Cada alumno deberá dominar la arquitectura global del programa y estar capacitado para exponer o justificar de forma general módulos elaborados por otros miembros del grupo.
- Oportunamente, el cuerpo docente dará a conocer la fecha límite requerida para la entrega de la nómina de integrantes y la elección del juego a desarrollar.

2. Requisitos del juego

- El desarrollo deberá estar realizado en lenguaje C. La lógica de funcionamiento del juego será implementada por el grupo y deberá aplicar los contenidos trabajados en la asignatura.
- El programa pedirá el nombre del jugador, lo saludará y presentará las reglas o instrucciones antes de jugar.
- El programa debe permitir completar una partida. Al finalizar, mostrará el ranking o un resumen de resultados/puntajes, según corresponda al juego elegido, y permanecerá en esa pantalla hasta que el usuario presione una tecla; luego finalizará. El criterio de puntaje lo definirá y documentará el grupo.
- Para los casos donde se implemente juego “contra la PC” y se requiera una decisión del sistema, no se exige una estrategia inteligente, sino un rival funcional que permita jugar con fluidez.
- Se admite una interfaz 100 % de consola, mediante texto. Como alternativa optativa se admite una interfaz gráfica; se sugiere SDL3, biblioteca para C. No se exige desarrollar las dos interfaces ni se otorgará calificación adicional por la apariencia gráfica.
- El uso de inteligencia artificial (IA) para generar código está autorizado exclusivamente en la capa gráfica opcional. El grupo deberá diseñar y comprender cómo se comunica la interfaz gráfica con la lógica, pero no se le exigirá comprender la implementación interna del código gráfico generado ni de SDL3. Consultar el Anexo A.



3. Juegos disponibles

- Póker (contra la PC)
- UNO (contra jugadores controlados por la PC)
- Ajedrez (dos jugadores humanos)
- Pong
- Sopa de letras (generar y resolver)
- Sudoku
- Memotest (encontrar pares).
- Truco (contra la PC)
- Buscaminas
- Ruleta
- Batalla Naval (dos jugadores humanos)
- Blackjack (contra la PC)
- Ahorcado
- Tetris
- Snake

Antes de empezar, cada grupo acordará con la cátedra el alcance concreto de su juego cuando existan distintas variantes o una complejidad considerable (por ejemplo, póker, ajedrez, Tetris o Sudoku).

4. Forma de entrega y documentación

La cátedra creará y asignará un repositorio de GitHub a cada grupo, con permisos de escritura para sus integrantes. Se recomienda utilizarlo durante el desarrollo, pero no es obligatorio: quienes trabajen por otros medios deberán cargar el proyecto completo en ese repositorio antes de la fecha límite.

Antes de la fecha y hora límite informadas por la cátedra, cada grupo deberá tener cargado en su repositorio el trabajo completo, incluido el código fuente, los recursos necesarios y el README.md. Se evaluará únicamente el material entregado dentro del plazo establecido.

- Se entregará el código fuente .c y .h, los recursos indispensables y las instrucciones para compilar y ejecutar el programa. El ejecutable solo no constituye una entrega suficiente.
- El archivo README.md será el único informe obligatorio. De manera breve, incluirá:
 - a. Integrantes y aportes/horas aproximadas por cada uno.
 - b. Reglas y alcance del juego.
 - c. Instrucciones para compilar, ejecutar y controlar el juego
 - d. Organización general de datos y funciones
 - e. Si hay interfaz gráfica, también indicará qué funciones conectan la interfaz con la lógica y qué parte gráfica se generó mediante IA.
- El README.md podrá incluir opcionalmente capturas de pantalla, imágenes, tablas o diagramas para facilitar la explicación. Se sugiere Mermaid para elaborar diagramas sencillos (por ejemplo, el recorrido de una jugada o la relación entre las partes del programa) directamente en el README.md, donde GitHub puede visualizarlos. Los archivos gráficos utilizados deberán formar parte del repositorio y estar enlazados desde el README.md.



5. Presentación grupal

Demostración grupal: hasta 8 minutos, distribuidos equitativamente. Mostrar una partida o secuencia representativa y explicar, de forma breve, cómo se organizaron los datos y las funciones.

6. Defensa oral individual

Luego habrá un coloquio individual de aproximadamente 3–4 minutos por integrante. Cada uno deberá poder responder preguntas como las que se presentan de referencia a continuación, aplicadas al programa efectivamente entregado.

Durante el coloquio se podrá pedir localizar una función, seguir manualmente una jugada, identificar un caso límite o proponer o realizar un cambio pequeño y acotado. Se evaluará la comprensión del proyecto desarrollado.

Preguntas orientativas:

Cada estudiante deberá poder responder sobre el juego completo, con especial profundidad en su contribución. Se formulará una selección de preguntas, no necesariamente todas.

- ¿Qué datos representan el estado actual de la partida? Señálos en el código y explicá cuándo cambian.
- ¿Por dónde empieza el programa? Mostrá el recorrido desde el menú/inicio hasta una jugada concreta.
- ¿Qué función decide si una jugada es válida? ¿Qué devuelve o qué cambia cuando es inválida?
- ¿Qué funciones elegiste separar y por qué? ¿Qué problema habría si las unieras en una sola?
- ¿Qué parámetros recibe una función X y qué resultado produce? ¿Por qué?
- ¿Cómo se determina el fin de la partida y el resultado? ¿Dónde se calcula el ranking?
- ¿Qué ocurre si el jugador ingresa una opción fuera de rango, repite una jugada o no quedan movimientos?
- ¿Qué parte del programa cambiarías para modificar una regla sin alterar el resto?
- Si aparece un resultado incorrecto, ¿cómo localizarías la función responsable y cómo lo probarías?
- ¿Qué decidió o programó cada integrante? Elegí una parte hecha por otra persona y explicá cómo se comunica con la tuya.
- Si utilizaron una interfaz gráfica generada con IA: ¿qué le pidieron, qué archivos o funciones integraron y cómo comprobaron que la interfaz usa correctamente la lógica? Si hicieron todo en consola: ¿cómo separaron la presentación de las reglas?
- Si hay gráficos: ¿qué acciones manda la interfaz a la lógica, qué datos recibe y qué código habría que tocar para cambiar solamente el aspecto visual?
- Si hay gráficos: ¿qué pasaría si mañana reemplazaran raylib o SDL3 por otra forma de mostrar el juego? ¿Qué funciones deberían permanecer sin cambios?



7. Criterios de evaluación

El programa y el código entregados constituyen el material sobre el que se demostrará lo aprendido, pero su funcionamiento, complejidad o apariencia no determinan por sí solos la calificación. El resultado de la evaluación se fundamentará principalmente en el informe y, especialmente, en la defensa individual: cada estudiante deberá demostrar que puede comprender, recorrer, explicar y justificar el código del proyecto (con excepción de gráficos si se usan), así como razonar sobre sus decisiones, errores y posibles modificaciones.

La cátedra confirmará la ponderación definitiva. Se considerarán:

1. Funcionamiento y cumplimiento de requisitos;
2. Organización del código, datos y funciones y validación de casos;
3. Distribución de tareas y participación del grupo;
4. Comprensión individual demostrada en la defensa. La comprensión individual tendrá peso propio.

No se considerará extra para la calificación una interfaz gráfica por su apariencia ni se penalizará una solución de consola. Si hay interfaz, se observará especialmente si el grupo definió y puede justificar la separación entre presentación y lógica.



ANEXO A

Guía de diseño y separación de responsabilidades

Esta guía orienta cómo pensar el programa antes de escribirlo. “Capa” significa aquí un conjunto de responsabilidades, no necesariamente la obligación de crear un archivo diferente por cada parte.

A.1. Preguntas para empezar

Antes de programar, respondan: ¿qué información hay que guardar durante una partida?, ¿qué funciones realizan las reglas?, ¿qué función recibe una acción del jugador?, ¿qué otras muestran información?, ¿cómo se sabe si la partida terminó?

A.2. Tres responsabilidades que deben distinguir

1. Datos / estado: tablero, cartas, puntaje, turno, intentos u otros valores necesarios para representar el juego.
2. Lógica del juego: inicializa una partida, verifica acciones, modifica los datos, calcula resultados y determina si la partida terminó.
3. Entrada y presentación: recibe teclas u otras acciones y muestra instrucciones, tablero, puntajes, mensajes y ranking.

La información del juego, sus reglas y la forma de mostrarlo deben estar organizadas de forma comprensible. Esquema orientativo: acción del jugador → verificar si es válida → actualizar el estado → consultar resultado → mostrar el nuevo estado. Este esquema se adapta a cada juego. El programa debe dividirse en funciones con tareas claras, evitar duplicación innecesaria y usar nombres e indentación comprensibles. La cantidad de archivos no constituye un mérito por sí misma: se valora que la separación tenga sentido.

A.3. Dónde está el límite entre lógica e interfaz

- La lógica conoce las reglas: decide si una acción es válida, modifica el tablero o las cartas, administra turnos y puntajes y establece cuándo termina la partida.
- La interfaz recibe una acción del usuario y muestra información. Puede traducir un clic del mouse a coordenadas de casilla, pero no debe decidir si la jugada vale, sumar puntos ni modificar por su cuenta los datos del juego.
- La lógica no debe necesitar incluir funciones gráficas o de interfaz con el usuario. Así se podrá modificar o reemplazar la presentación sin reescribir las reglas.

Ejemplo — Memotest: el jugador selecciona la fila 2, columna 3. La interfaz comunica esa selección; una función de lógica comprueba si puede destaparse, actualiza los datos y devuelve el resultado necesario para mostrar las cartas visibles, el número de intentos y los aciertos. Si la jugada es inválida, la lógica lo informa y la interfaz muestra un mensaje. La pantalla no decide qué cartas forman pareja.

A.4. Definir el contrato entre ambas partes

Antes de construir o pedir a una IA la interfaz, describan en unas pocas líneas:

1. Qué necesita mostrar: por ejemplo, dimensiones y contenido visible del tablero, turno, puntaje y mensajes. No exponer información que el jugador no debe conocer.



2. Qué acciones puede comunicar: por ejemplo, “seleccionar casilla(fila, columna)”, “iniciar partida” o “salir”.
3. Qué funciones de C se usarán para intercambiar esos datos y acciones: nombre, parámetros y resultado esperado, aunque sea mediante un listado o esquema sencillo.
4. Qué sucede ante acciones inválidas y quién controla el inicio, la repetición de turnos y el final de la partida.

Ejemplo de contrato — Memotest (funciones y valores ilustrativos):

1. Capa de entrada y presentación (consola o gráfica): recibe del jugador la fila y la columna seleccionadas. Llama a la función de lógica seleccionar_casilla(fila, columna) y recibe su resultado: 1 si la acción es válida o 0 si es inválida. Muestra el mensaje correspondiente, pero no decide las reglas ni modifica las cartas.
2. Capa de lógica del juego: int seleccionar_casilla(int fila, int columna) recibe las coordenadas desde la interfaz, comprueba si la acción está permitida, actualiza los datos internos cuando corresponde y devuelve 1 o 0 a la interfaz. int obtener_casilla_visible(int fila, int columna) recibe una posición y devuelve a la interfaz -1 si la carta debe permanecer oculta, o un número que identifica la carta que puede mostrarse. int obtener_intentos(void) devuelve el total de intentos; int partida_terminada(void) devuelve 1 si terminó o 0 si continúa. Estas funciones de consulta no cambian las reglas ni revelan cartas ocultas.

Recorrido de una jugada: jugador → interfaz (captura fila y columna) → seleccionar_casilla de la lógica (valida y actualiza) → interfaz (recibe 1 o 0, consulta las casillas visibles, los intentos y el fin de la partida) → pantalla. Las flechas indican quién llama a quién y qué información vuelve. La lógica no dibuja ni solicita teclas; la interfaz no calcula parejas ni puntajes. Los nombres y valores de retorno son ejemplos: cada grupo definirá y explicará su propio contrato con los recursos de C trabajados en clase, sin necesidad de crear una biblioteca compleja.

A.5. Cómo encargar una interfaz gráfica a una IA

Si eligen SDL3, pueden generar íntegramente el código de esa capa con IA. Primero deben tener definidas y probadas las reglas y el contrato anterior. En el pedido a la IA indiquen:

1. *“Construí únicamente la interfaz de este juego en C con SDL3. Estas son las funciones disponibles y sus parámetros: [contrato del grupo]. Estas son las acciones del usuario y los datos visibles: [descripción].”*
2. *“No escribas ni modifiques funciones de reglas, estado, puntaje o resultados. No inventes funciones: si falta una operación en el contrato, indicá qué falta antes de generar código.”*
3. *“Ubicá la presentación en archivos diferenciables, explicá cómo compilarla y cómo se conecta con las funciones proporcionadas. No dupliques dentro de la interfaz las decisiones que toma la lógica.”*

Después de generar la interfaz, el grupo deberá integrarla, compilar y probar que una acción válida, una inválida y el final de partida producen el resultado esperado. En la defensa se preguntará por las funciones y datos que conectan ambas partes, no por los detalles internos del renderizado o la gestión de ventanas de SDL3.

Referencia oficial de SDL3 (documentación y ejemplos): <https://wiki.libsdl.org/SDL3/FrontPage> · <https://examples.libsdl.org/>



ANEXO B

Uso básico de GitHub y Git

Un repositorio de software es un espacio organizado que reúne los archivos de un proyecto (código fuente, documentación y recursos). Cuando utiliza Git, también conserva el historial de versiones, lo que permite conocer los cambios y recuperar versiones anteriores. Un repositorio puede estar en una computadora o alojado en Internet y Git es el sistema que registra esas versiones; GitHub es una plataforma en Internet que permite alojar repositorios Git, compartirlos y colaborar con otras personas.

Para este trabajo, la cátedra creará un repositorio en GitHub para cada grupo y dará acceso a sus integrantes. Ese repositorio será el lugar de entrega obligatorio: antes de la fecha límite deberá contener el código, los recursos necesarios y el README.md.

Hay dos modalidades de uso posibles en el marco del desarrollo y entrega del trabajo integrador, las que se abordarán en esta guía de referencia:

- B.1 Explica cómo subir los archivos desde el navegador, sin instalar Git, y utilizar la plataforma solo para la entrega final.
- B.2 explica cómo utilizar Github, Git y Visual Studio Code (VS Code) para desarrollar en equipo sobre el mismo repositorio. Esta segunda modalidad se recomienda, pero es opcional.

B.1. Uso mínimo: entregar el trabajo desde el navegador

1. Crear una cuenta gratuita en <https://github.com/signup>, informar el nombre de usuario a la cátedra y aceptar la invitación al repositorio del grupo. Abrir el enlace proporcionado por los docentes.
2. Preparar el proyecto: archivos fuente .c y .h, README.md y recursos indispensables. Mantener carpetas comprensibles. No subir solamente un ejecutable o un ZIP que oculte los fuentes.
3. En el repositorio, elegir «Add file» → «Upload files». Arrastrar archivos o una carpeta, o utilizar el selector de archivos. Revisar la lista y las rutas antes de continuar. GitHub permite cargar hasta 100 archivos simultáneamente y un máximo de 25 MiB por archivo mediante el navegador; si el proyecto supera esos límites, consultar a la cátedra.
4. Escribir un mensaje breve, como «Entrega final», y confirmar con «Commit changes» (o la opción equivalente). Seleccionar archivos no alcanza: es necesario confirmar la operación. Si no pueden incorporar los cambios directamente, avisar a la cátedra.
5. Para crear o corregir el README.md, abrirlo en GitHub, usar el ícono de lápiz y confirmar los cambios. El README.md constituye el informe solicitado en la sección.
6. Regresar a la página principal del repositorio y verificar que estén visibles todos los archivos, las carpetas y el README.md actualizado antes del plazo límite.

Documentación oficial:

Cargar archivos:

<https://docs.github.com/es/repositories/working-with-files/managing-files/adding-a-file-to-a-repository>

Editar archivos:

<https://docs.github.com/es/repositories/working-with-files/managing-files/editing-files>



B.2.: trabajar en equipo desde VS Code

Git registra versiones en cada computadora y GitHub conserva el repositorio compartido. Esta modalidad no requiere comandos de terminal, pero sí instalar Git. Se propone comenzar con una sola rama, main: alcanza para grupos pequeños si se organizan y evitan editar simultáneamente las mismas líneas.

Preparación inicial de cada integrante: instalar Git (<https://git-scm.com/downloads>) y VS Code (<https://code.visualstudio.com/>), aceptar la invitación y autenticarse en GitHub cuando VS Code lo solicite. No crear otro repositorio: utilizar el asignado por la cátedra.

1. Clonar una sola vez. En GitHub, abrir el repositorio y copiar su dirección mediante «Code» → HTTPS. En VS Code, abrir «Control de código fuente» (Source Control), seleccionar «Clone Repository» o «Git: Clone», pegar la dirección, elegir dónde guardarlo y abrir esa carpeta. Cada compañero hace lo mismo en su PC. Si el juego ya estaba empezado, copiar sus archivos dentro de la carpeta clonada; no borrar la carpeta oculta .git.
2. Coordinar el trabajo. Distribuir inicialmente tareas y, cuando sea posible, archivos distintos. Avisar antes de cambiar un archivo compartido y acordar quién integra modificaciones importantes. No es obligatorio crear ramas ni solicitudes de incorporación (pull requests) para esta actividad.
3. Antes de empezar una sesión, con el trabajo local previo guardado y confirmado, seleccionar «Pull» en VS Code para recibir lo que subieron los compañeros. Revisar los cambios. También hay que hacer Pull cuando un compañero modificó archivos desde el navegador.
4. Programar, guardar y probar. Abrir «Control de código fuente», examinar qué archivos cambiaron y seleccionar los correspondientes con «Stage Changes» (ícono +). No incorporar ejecutables ni carpetas de compilación.
5. Escribir un mensaje que describa el avance y seleccionar «Commit». Esto registra los cambios localmente: todavía no implica que estén en GitHub. Después elegir «Sync Changes» (sincronizar: traer cambios remotos y subir commits locales), o «Push» si el repositorio ya está actualizado. Entrar a GitHub y verificar la publicación.
6. El resto del equipo hace Pull para recibir la versión nueva. Repetir este circuito simple: Pull → modificar y probar → revisar → Commit → Sync Changes. Git no sincroniza por sí solo archivos locales sin confirmar ni subir.

Si aparece un conflicto, falla un Push o hay un aviso que no se comprende, detenerse y pedir ayuda sin sobrescribir trabajo ajeno. Antes de la entrega, comprobar en la web de GitHub que el código completo y el README.md estén publicados.

Documentación oficial:

Guía rápida de Git en VS Code:

<https://code.visualstudio.com/docs/sourcecontrol/quickstart>

Clonar y sincronizar repositorios:

<https://code.visualstudio.com/docs/sourcecontrol/repos-remotes>

Resolver conflictos:

<https://code.visualstudio.com/docs/sourcecontrol/merge-conflicts>

Conceptos básicos de Git:

<https://docs.github.com/es/get-started/using-git/about-git>



ANEXO C

Qué es el README.md y cómo usar Markdown

C.1. El README.md: presentación e informe del proyecto

README significa «léeme». Es un archivo de texto que explica qué hace un proyecto y cómo utilizarlo. Cuando está en la carpeta principal (raíz) del repositorio, en plataformas como GitHub se muestra su contenido en la página inicial. Para este trabajo, README.md constituye el informe obligatorio indicado en la sección 4 (no hay que entregar un Word o PDF adicional). Debe ayudar a otra persona a entender el juego, compilarlo, ejecutarlo y conocer cómo está organizado. Es conveniente redactarlo durante el desarrollo y revisarlo antes de la entrega.

C.2. ¿Qué significa la extensión .md?

.md indica que el archivo usa Markdown: texto común con unos pocos signos para darle formato. No es código fuente ni exige instalar un programa especial. Se puede escribir desde el navegador de GitHub o desde VS Code; GitHub convierte las marcas en títulos, listas, enlaces, imágenes y bloques de código al mostrar el README.

Marcas habituales:

*Mi juego* → *título principal*.

*Cómo jugar* → *subtítulo*.

****Importante**** → *texto en negrita*.

- *Elegir una casilla* → *elemento de una lista*.

``iniciar_partida()`` → *nombre de función o código dentro de una oración*.

`[Manual](https://ejemplo.org)` → *enlace a una página web*.

`![Pantalla del juego](imagenes/pantalla.png)` → *imagen guardada dentro del repositorio, mediante una ruta relativa*.

Para mostrar varias líneas de código o instrucciones de compilación, se usan tres acentos graves al principio y al final de un bloque; se puede escribir `c` después de los tres primeros para indicar el lenguaje C. También pueden utilizarse tablas y, opcionalmente, diagramas Mermaid directamente en el README.

C.3. Estructura inicial sugerida

Pueden empezar con «# Nombre del juego» y agregar apartados «## Integrantes y aportes», «## Reglas y controles», «## Compilación y ejecución», «## Organización de datos y funciones» y, solo si corresponde, «## Interfaz gráfica». Estos encabezados son ejemplos de organización; el contenido exigido es el de la sección 4.



C.4. Cómo editarlo y comprobar el resultado

En GitHub: abrir README.md, seleccionar el ícono de lápiz, escribir el texto, usar la vista previa («Preview») para revisar el formato y confirmar con «Commit changes».

En VS Code: editar y guardar README.md, abrir su vista previa de Markdown y luego confirmar y subir el cambio siguiendo el Anexo B. Comprobar siempre el resultado final en la página del repositorio.

Documentación oficial:

Sobre README:

<https://docs.github.com/es/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

Sintaxis Markdown:

<https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>